



Estructuras de Datos

Estudio de Caso I

Estudiante

Andrés Gabriel Meléndez Carvajal

Profesor:

Romario Salas Cerdas

Fecha:

21 junio, 2026

Estudio de Caso I

Aplicación de pilas en el análisis de expresiones aritméticas.

Enlace al repositorio: https://github.com/melendez17/PrimerEstudioCaso_EstDatos

Pilas en sistemas aritméticos

Investigando un poco según menciona GeeksforGeeks (2023) el análisis de expresiones aritméticas es una de las aplicaciones más clásicas de las pilas de software en un sistema. Esta funcionalidad es la base de los compiladores, intérpretes de lenguajes de programación, calculadoras científicas, hojas de cálculo y muchas otras cosas que seguimos usando a diario, ya que cualquier programa que deba evaluar una expresión matemática escrita por un usuario necesita en algún punto procesarla mediante una pila.

Pero primero, ¿a qué nos referimos cuando hablamos de una expresión aritmética? En este caso cuando hablamos de una expresión aritmética podemos decir que está formada por 4 tipos de tokens:

1. Números literales como 3, 4, 5, etc.
2. Variables numéricas que al final van a ser como indicadores que representen valores “total”, “preciodeproducto”, etc
3. Operadores que van a ser los simbolos que indiquen la operación matemática
4. Paréntesis que nos van a alterar el orden de la evacuación como en cualquier otra expresión aritmética que hagamos a mano.

Que al final de cuentas todo eso junto nos daría algo como:

$$(3 + x) * (5 - y)$$

Ahora bien uno de los grandes desafíos que tenemos en este problema es que los humanos escribimos expresiones en notación infija, es decir que el operador va a ir entre los operandos. Para comprender un poco más porque nos será útil más tarde podemos ver las siguientes notaciones que como bien explica Gupta (2024):

- Infija que es lo que naturalmente hacemos, como: $a + b$

- prefija que el operador va antes que los operandos: $+ a b$
- postfija en la que el operador va después de los operandos: $a b +$

Por ejemplo si ponemos: $a + b$, esa anotación es ambigua para la computadora y sin reglas de prioridad y en este caso los compiladores necesitan una representación sin ambigüedad y es ahí donde una pila es sumamente fundamental.

¿Por qué la pila es la mejor opción?

Primero debemos entender la lógica clara de cómo se puede analizar una expresión. Por ejemplo: Cuando nosotros analizamos una expresión, la recorremos de izquierda a derecha, cuando se encuentra un operador no siempre se puede aplicar inmediatamente, ¿por qué? Porque puede haber operadores de mayor precedencia después o pueden aparecer paréntesis que cambien el orden de la operación es por esto que el sistema necesita de algún modo recordar los operadores encontrados y aplicarlos en el momento correcto no inmediatamente.

Bajo esa dinámica y lógica, sigue exactamente el principio LIFO que es el conocido “Last in first out” que podemos decir que define las pilas. En este caso la pila nos garantiza que los operadores siempre sean procesados en el orden correcto según su precedencia y asociatividad.

Pero cuál dinámica? ¿Cuál lógica? Este va a ser el algoritmo de Dijkstra “Shunting-yard” que lo que busca es convertir una expresión matemática en notación infija ($3+4$) que es la que nosotros utilizamos, en una postfija ($3 4 +$) que es la que utilizan las máquinas, todo esto utilizando una pila.

Según un artículo de Brilliant (2026) este algoritmo se le conoce de esa manera porque se asemeja a un patio de maniobras de trenes, ¿por qué? podemos hacer la siguiente analogía:

- Una vía de entrada que es por donde viene el tren original es como nuestro input, en el que recibimos la expresión aritmética original ($4+2$) resulta que cada número y operador es una “vagon” es decir en este caso tenemos 3 vagones.

- Tenemos también una vía de salida que sería nuestro output y donde comúnmente los trenes se van ya transformados y con los vagones en el orden que los necesitan, en nuestro caso sería la expresión ya postfija (4 2 +).
- Pero aquí viene lo interesante, existe una vía de desvío o una vía muerta, es una vía secundaria donde los vagones pueden entrar pero solo pueden salir en orden inverso (last in, first out) es decir lo que conocemos exactamente como una pila.

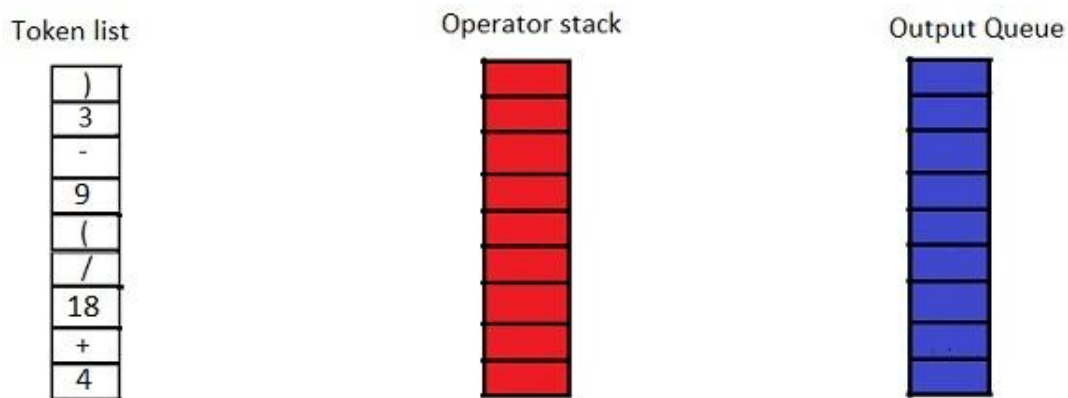
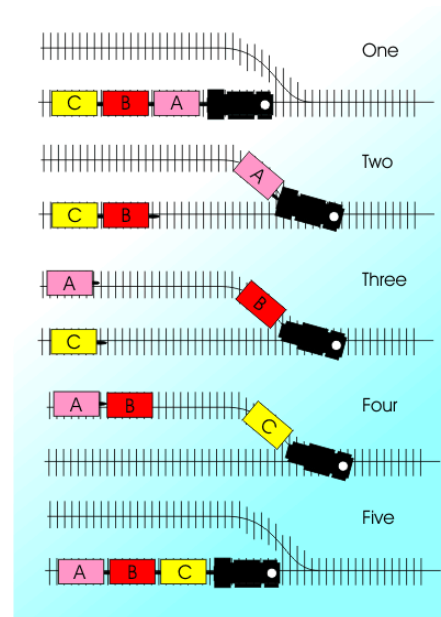


Imagen extraída de Brilliant - Shunting Yard Algorithm

Que también lo podemos ver como “Vía de entrada”, “Vía de desvío”, “Vía de salida”

Es por esto que una pila es la mejor opción, una cola por ejemplo procesaría los operadores en el orden de llegada, ignorando la precedencia y es aquí donde la pila nos sirve mucho ya que almacena temporalmente con acceso al elemento más reciente y las reglas del algoritmo cumplen lo siguiente:

1. Si el token es un número o variable, pasa directamente a la salida
2. Si es un operador, pasan a la pila (o se sacan los operadores con mayor precedencia antes de apilarlo)
3. Si es un **(**, se apila directamente como marcador

4. Si es un **)**, se vacía la pila hasta encontrar el **(** correspondiente, que se descarta.
5. Al terminar la entrada, se vacía el resto de la pila de salida

Seguridad, orden y eficiencia

Comentario sobre la seguridad, orden y eficiencia que una pila aporta a la realización de la funcionalidad seleccionada.

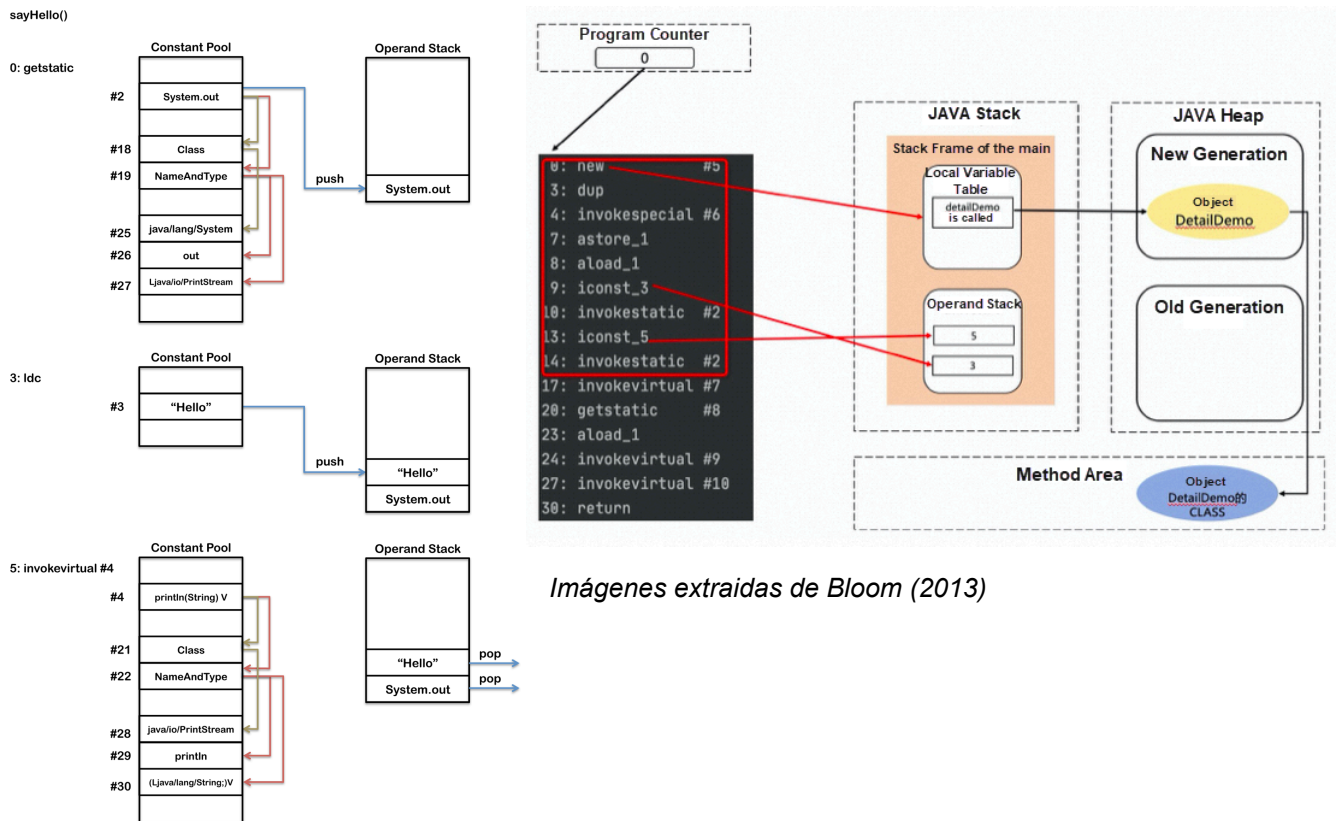
Una de las cosas más importantes a nivel de seguridad es que solo usamos operaciones ya muy bien definidas de la pila como el push, pop, peek, isEmpty y de esta manera el sistema nunca accede a un operador que no debe, aportando también al orden ya que si hay algo faltante, nos daremos cuenta con una condición de error que se puede detectar fácilmente ya que la pila quedaría con elementos de más o el algoritmo intenta extraer de una pila vacía.

Las operaciones fundamentales de una pila (push y pop) tienen una complejidad temporal de $O(1)$, lo que significa que el tiempo de ejecución es constante e independiente del volumen de datos. Es por esto que el algoritmo Shunting-Yard según menciona el departamento de matemáticas y ciencias computacionales de oxford tiene complejidad temporal $O(n)$, donde n es el número de tokens. Cada token es procesado exactamente una vez, y cada operador entra y sale de la pila exactamente una vez.

Ejemplo en software de la funcionalidad

El compilador de Java ya que el compilador no puede enviar simplemente lo que recibe, tiene que enviar una expresión ya analizada donde se haya determinado los operandos, los operadores, los paréntesis y las precedencias y para esto se utilizan en muchos algoritmos las pilas como el que acabamos de mencionar.

El compilador de Java (javac) no genera instrucciones orientadas a registros. En su lugar, traduce esta expresión infija en una secuencia pura de instrucciones LIFO destinadas a la denominada Pila de Operandos (Operand Stack) de la JVM.

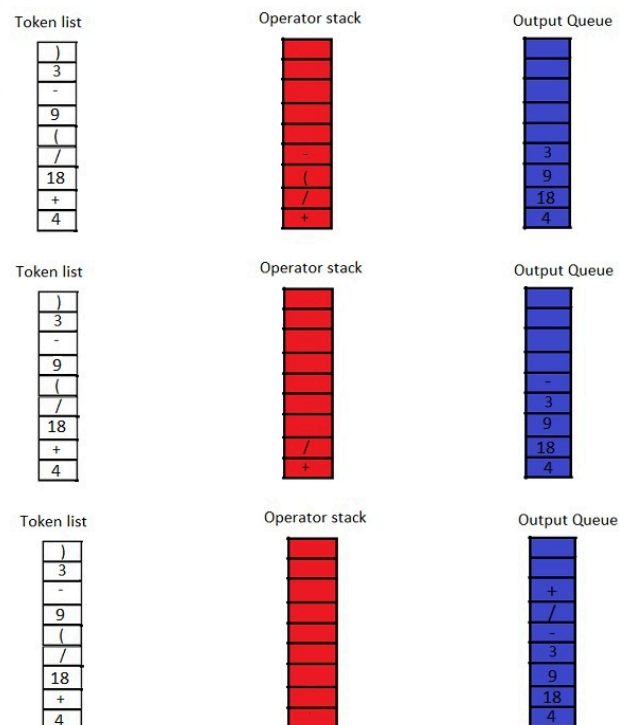
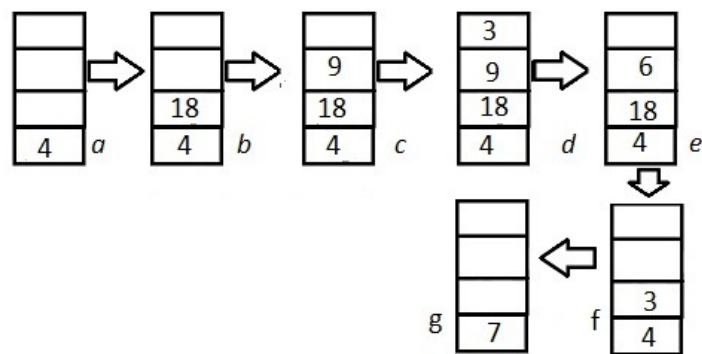


Imágenes extraídas de Bloom (2013)

Representación gráfica de las operaciones

1. Si el símbolo entrante es un operando, lo imprimo directamente en la salida.
2. Si el símbolo entrante es un paréntesis izquierdo (, lo agrego a la pila.
3. Si el símbolo entrante es un paréntesis derecho), lo descarto como símbolo de entrada y extraigo e imprimo los elementos de la pila hasta encontrar un paréntesis izquierdo (. Luego extraigo ese paréntesis izquierdo y lo descarto.
4. Si el símbolo entrante es un operador y la pila está vacía o tiene un paréntesis izquierdo en la cima, inserto el operador en la pila.
5. Si el símbolo entrante es un operador y tiene mayor precedencia que el operador ubicado en la cima de la pila, o tiene la misma precedencia pero es asociativo por la derecha, o la pila está vacía, o la cima contiene un paréntesis izquierdo (, agrego el operador a la pila.

6. Si el símbolo entrante es un operador y tiene menor precedencia que el operador ubicado en la cima de la pila, o tiene la misma precedencia pero es asociativo por la izquierda, continúo extrayendo e imprimiendo operadores de la pila hasta que esa condición deje de cumplirse. Después, inserto el operador entrante en la pila.
7. Cuando llego al final de la expresión, extraigo e imprimo todos los operadores restantes en la pila. En este punto no deben quedar paréntesis almacenados.



Imagenes extraidas de Brilliant (2026)

Ejemplo del algoritmo de shunting yard

Paso 1	Input	$4 + 18 / (9 - 3)$	Símbolo analizado	4
	Pila		Fila	4
Paso 2	Input	$+ 18 / (9 - 3)$	Símbolo analizado	"+"
	Pila		Fila	4
	"+"			
Paso 3	Input	$18 / (9 - 3)$	Símbolo analizado	18
	Pila		Fila	4 18
	"+"			
Paso 4	Input	$/ (9 - 3)$	Símbolo analizado	"/"
	Pila		Fila	4 18
	"/"			
	"+"			
Paso 5	Input	$(9 - 3)$	Símbolo analizado	"("
	Pila		Fila	4 18
	"("			
	"/"			
	"+"			
Paso 6	Input	$9 - 3$	Símbolo analizado	9
	Pila		Fila	4 18 9
	"("			
	"/"			
	"+"			
Paso 7	Input	$- 3$	Símbolo analizado	"-"
	Pila		Fila	4 18 9
	"-"			
	"("			
	"/"			
	"+"			
Paso 8	Input	$3)$	Símbolo analizado	3
	Pila		Fila	4 18 9 3
	"-"			
	"("			
	"/"			
	"+"			
Paso 9	Input	$)$	Símbolo analizado	")"
	Pila		Fila	4 18 9 3 -
	"/"			
	"+"			
Paso 9	Input		Símbolo analizado	
	Pila		Fila	4 18 6 /
	"+"			
Paso 9	Input		Símbolo analizado	
	Pila		Fila	4 3
	"+"			
Paso 9	Input		Símbolo analizado	
	Pila		Fila	4 3 +
Paso 9	Input		Símbolo analizado	
	Pila		Fila	7
Resultado final:				7

Explicación:

Comenzamos analizando el primer valor de izquierda a derecha, como es número, se agrega directamente a la fila.

Explicación:

Analizamos el segundo valor que es el "+", al ser símbolo se agrega directamente a la pila, como está vacía no hay que ejecutar nada y solo lo agregamos

Explicación:

Analizamos el siguiente valor que es 18, al ser número se agrega directamente a la cola.

Explicación:

El siguiente valor es "/", por lo que debe ir a la pila. La primera validación que hacemos en la pila es está vacía? Como no está vacía (tiene el +) entonces valoramos si el / tiene prioridad o no, al / tener más prioridad, simplemente lo agregamos a la pila.

Explicación:

El siguiente valor es un "(" por lo que simplemente lo agregamos a la pila, el parentesis que abre simplemente se agrega, si fuera cierre sería distinto.

Explicación:

El siguiente valor es 9 por lo que lo agregamos a la fila.

Explicación:

El siguiente es un menos, por lo que al ser símbolo lo agregamos a la pila

Explicación:

El siguiente valor es 3 por lo que lo agregamos a la fila.

Explicación:

Acá viene lo divertido, el siguiente valor que estamos analizando es ")" ahora mencioné que si fuera un parentesis cerrado sería distinto, cuál sería el proceso? Vamos a extraer de la pila todos los valores hasta que encontremos la apertura del parentesis y eliminamos los dos parentesis y agregamos los símbolos a la fila "-"

Explicación:

Realizamos la operación con el símbolo que acabamos de agregar que fue el menos y lo hacemos con los últimos 2 valores procedentes que eran 9 y 3 por lo tanto se realizó la operación $9 - 3$ - que en infija sería $9 - 3 = 6$

Explicación:

Luego realizamos la operación del / que sería $18 / 6$ en infija y postfija sería $18 6 /$ que es como lo leemos en la salida. Y el resultado nos daría 3

Explicación:

Extraemos el último símbolo que era el de la suma y lo realizamos en la fila con los últimos valores que nos hacen falta que sería en post fijo $4 3 +$ y en infijo $4 + 3$.

Explicación:

Después de todo el proceso así es como llegaríamos a el resultado final que en este caso era 7

Ejemplo del algoritmo de shunting yard

Paso 1	Input	(2 + 4) * (4 + 6)	Símbolo analizado	(
	Pila		Fila	
				(

Explicación:
Comenzamos analizando el primer valor de izquierda a derecha, como es símbolo, se agrega directamente a la pila.

Paso 2	Input	2 + 4) * (4 + 6)	Símbolo analizado	2
	Pila		Fila	2

Explicación:
Analizamos el segundo valor que sería 2, por lo que lo agregamos directamente a la fila

Paso 3	Input	+ 4) * (4 + 6)	Símbolo analizado	+
	Pila		Fila	2
				+
				(

Explicación:
Como es símbolo lo añadimos a la pila

Paso 4	Input	4) * (4 + 6)	Símbolo analizado	4
	Pila		Fila	2
				4
				+
				(

Explicación:
Número se agrega a la fila

Paso 5	Input	) * (4 + 6)	Símbolo analizado	)
	Pila		Fila	2
				4
				+
				(
				<- Se elimina

Explicación:
Es un parentesis cerrado por lo que se debe eliminar este parentesis y sacar de la pila todos los símbolos hasta encontrar el parentesis abierto y eliminarlo también

Paso 6	Input	* (4 + 6)	Símbolo analizado	*
	Pila		Fila	2
				4
				+
				*

Explicación:
El siguiente valor es símbolo, va a la pila

Paso 7	Input	(4 + 6)	Símbolo analizado	(
	Pila		Fila	2
				4
				+
				*
				(

Explicación:
El siguiente valor es un parentesis, osea símbolo, va a la pila

Paso 8	Input	4 + 6)	Símbolo analizado	4
	Pila		Fila	2
				4
				+
				4
				*
				(

Explicación:
El siguiente es número, por lo que va a la fila

Paso 9	Input	+ 6)	Símbolo analizado	+
	Pila		Fila	2
				4
				+
				4
				*
				(
				+

Explicación:
El siguiente es símbolo así que va a la pila

Paso 9	Input	6)	Símbolo analizado	6
	Pila		Fila	2
				4
				+
				4
				6
				*
				(
				+

Explicación:
El siguiente es número, va a la fila

Paso 9	Input	)	Símbolo analizado	)
	Pila		Fila	2
				4
				+
				4
				6
				+
				*
				(
				+
				<- Se elimina

Explicación:
Otra vez un cierre de parentesis por lo que lo eliminamos y sacamos todo de la pila hasta llegar a la apertura del parentesis y eliminarlo también.

Paso 9	Input		Símbolo analizado	
	Pila		Fila	2
				4
				+
				4
				6
				+
				*

Explicación:
Al ya no haber nada en el input, sacamos los últimos símbolos que hayan en la fila

Expresión Final	2	4	+	4	6	+
	2	4	+	4	6	+
	6	4	6	+	*	
	6	10	*			
Resultado final	60					

Explicación:
Una vez que tenemos la expresión final la hacemos de izquierda a derecha

Bibliografía

GeeksforGeeks. (2023, 19 junio). *Arithmetic Expression Evaluation*. GeeksforGeeks.

<https://www.geeksforgeeks.org/dsa/arithmetic-expression-evaluation/>

Shunting Yard Algorithm | *Brilliant Math & Science Wiki*. (2026).

<https://brilliant.org/wiki/shunting-yard-algorithm/>

Gupta, H. (2024). *Notaciones de expresiones: notación infija, prefija y postfija simplificadas*.

Medium.

<https://medium.com/@devharshgupta.com/understanding-expression-notations-infix-prefix-and-postfix-simplified-526073df55f5>

An illustration of JVM and the Java Program operation principle. (n.d.). Alibaba Cloud

Community.

https://www.alibabacloud.com/blog/an-illustration-of-jvm-and-the-java-program-operation-principle_600307

Bloom, J. D. (2013, November 24). *JVM Internals*. James D Bloom.

<https://blog.jamesdbloom.com/JVMInternals.html>

pikuma. (2024, December 26). *Shunting yard algorithm* [Video]. YouTube.

<https://www.youtube.com/watch?v=ceu-7gV1wd0>